Agentic AI for Economic Research

From JARVIS to your own second brain

Marco Le Moglie, Catholic University of the Sacred Heart
Four lessons of two hours, autumn 2026
GitHub repo

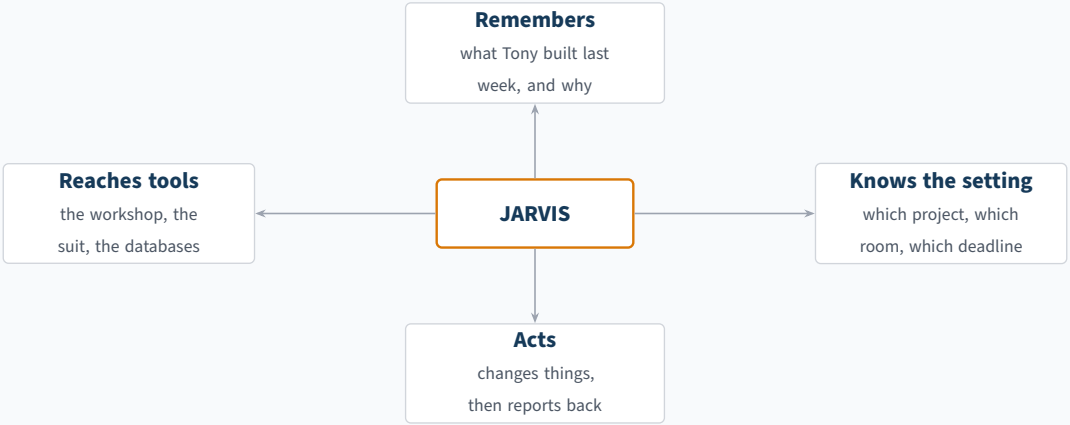
From JARVIS to the second brain

What we want from an assistant, and why memory is the missing piece

Can we build something like this today?

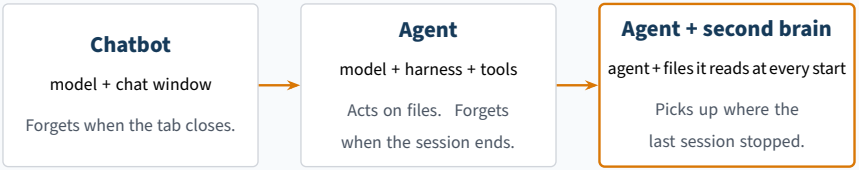
Iron Man (2008), Marvel Studios. The film is the reference. No still is embedded: this repository is public, and no frame with a licence for redistribution was found.

What makes JARVIS different from a chatbot?



A chatbot answers a message. JARVIS keeps working on the same project.

Can we build it today?



Feasible in 2026

Read and write project files. Call tools through MCP. Reload rules and state at every start. Load procedures only when needed.

Still missing

Nothing is remembered unless someone writes it down. The agent does not know what is worth keeping. A wrong note stays wrong until someone checks it.

Source: Lesson 1, model + harness = agent; Anthropic (2025).

The problem with ordinary chatbots

Six months of a paper in chat windows

Illustrative example.

Week 2: you explain the sample and the treatment again.

Week 5: the agent clusters by province. You had decided on municipality.

Week 9: a new chat. Why 2020 was dropped is gone.

Week 14: two chats give two different rules for outliers.

Week 20: the coauthor's instructions sit in a thread you cannot find.

Problem	What it costs
Repeated context	tokens and time at every start
Lost decisions	the same choice made twice, differently
Hard restarts	the first twenty minutes spent rebuilding
Inconsistent instructions	outputs that contradict each other
Fragmentation	no single place to check

A chat log records what was said. It is a poor record of what was decided.

Persistent memory

What to keep	In one line
State	where the project stands today
Decisions	what was chosen, when, and why
Constraints	what must not change
Sources	where each number comes from
Solved errors	the bug and the fix that worked
Next actions	the first thing to do next time
Procedures	steps you repeat, written once

Chat history

Everything said, in order. Long, and mostly noise for the next session.

Document archive

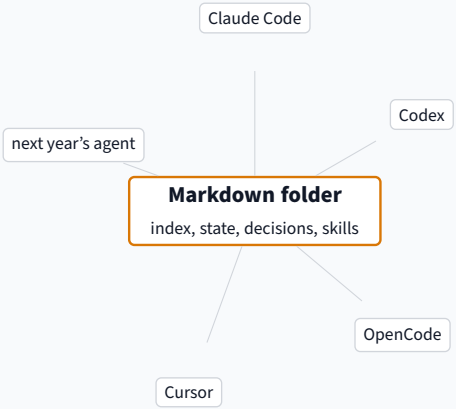
Papers, data, drafts. The evidence, not the summary.

Persistent memory

Short files, kept current by you or the agent, read at every start.

Memory is what you would tell a coauthor who joins tomorrow.

Own your knowledge, not your model



Source: AGENTS.md (2026); Anthropic (2026b); Cursor (2026).

Portable

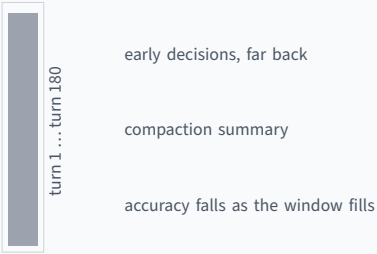
Plain text files. Any agent that can open a folder can use them. Changing provider does not move your memory.

Less portable

Rules files have different names: `AGENTS.md`, `CLAUDE.md`, `.cursor/rules`. MCP servers and plugins are set up again in each harness. Memory stored inside a vendor's app stays with the vendor.

New session, same knowledge

One long chat



New session + bootstrap

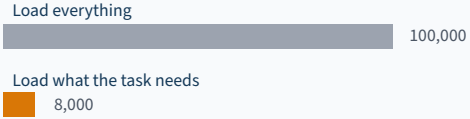
- rules file
- INDEX
- CURRENT_STATE
- NEXT_ACTIONS
- room to work

A fresh session helps when the chat has wandered or the summary has lost detail. Anything the closeout did not write down is gone.

A new session costs little if the files are current.

Source: Hong et al. (2025); Anthropic (2025).

Fewer tokens, better context



92% fewer input tokens for this request.

Illustrative numbers, not a measured benchmark.

Progressive disclosure	names and descriptions first, full text on demand
Selective loading	one project's files, not the whole vault
Retrieval	the few passages that match the question
Prompt caching	a repeated prefix billed at a discount
Compaction	old turns replaced by a summary

Savings per request are not total savings. Writing the notes, building an index, and keeping both current also cost tokens and time.

Source: Anthropic (2026a); OpenAI (2026b).

Portable skills

```
---
name: check-synthetic-panel
description: Check the synthetic census-tract panel for the transit-jobs project. Use when asked to check the panel, validate the dataset,
---

# Skill: check the synthetic panel

## When to use
The user asks to check, validate, or audit the transit-jobs panel.

## Input
areas/Research/transit-jobs/data/synthetic_panel.csv
```

Opening of the demo skill `check-synthetic-panel`. The agent sees the name and description first.

Source: Agent Skills (2026); Anthropic (2026c); OpenAI (2026a).

At start, the agent sees each skill's name and description: a few dozen tokens.

When a request matches, it reads the full `SKILL.md`.

Scripts, templates and references load only if the steps call for them.

The same folder works in any harness that follows the Agent Skills format.

The saving comes from loading in stages. Pasting a folder of procedures into every prompt saves nothing.

From chatbot to research assistant

Hypothesis	Identification	Data	Code	Results	Decisions
New metro stations raise employment within 1 km.	Staggered openings; Callaway and Sant'Anna estimator.	Synthetic panel, 400 tracts, 2010 to 2022.	`code/02_did.py` seed 2026.	Simulated ATT 0.031 (s.e. 0.012). Labelled as simulated.	14 Sep 2026: cluster by district, not tract.

Coauthor rules: never edit `data/raw`. Tables only through `make_tables.py`.

> **A real case**
>
> My mafia-culture state file says that 1,949 rows are the old artist universe, not 1,949 verified narcocorrido artists. An agent that reads it does not report the wrong number.

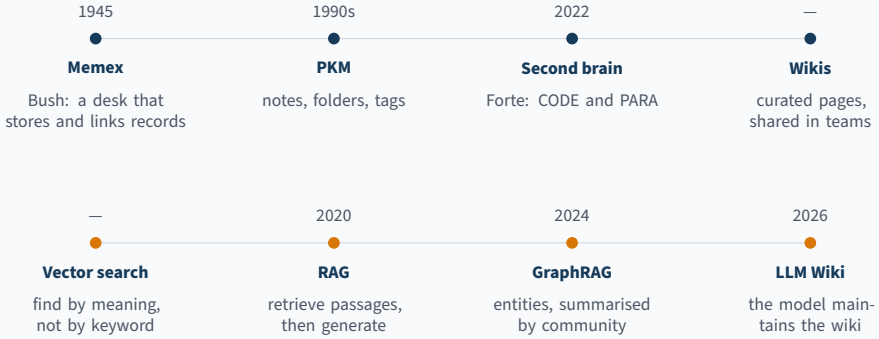
Each session starts from the last verified state.

Source: Synthetic demo; mafia-culture state file, 18 Oct 2026.

How AI memory can be designed

Prompt, retrieval, wiki, graph: what each design keeps

A short history of external memory



These layers coexist. A 2026 system often combines three of them.

Source: Bush (1945); Forte (2022); Lewis et al. (2020); Edge et al. (2024); Karpathy (2026).

Everything in the prompt



Strengths

Simple: no index, no pipeline.

The model reads every source in the author's own words.

Works while the material fits comfortably in the window.

Limits

The window caps the corpus.

Every request pays for every token.

Irrelevant pages dilute attention.

Every update means sending everything again.

Fine for one paper. Not for a research agenda.

Retrieval-augmented generation



Lexical: matches words (BM25). Good for names, codes, variable labels.

Semantic: matches meaning through embeddings. Good for paraphrase.

Hybrid: runs both and merges the rankings.

Failure	What happened
Wrong chunk	the split cut the key sentence
Missed source	the right document never ranks
Incomplete context	the answer needs ten passages; k is five
Unfaithful synthesis	the model writes past what the passages say

Source: Lewis et al. (2020); Robertson and Zaragoza (2009).

Knowledge bases, wikis and GraphRAG

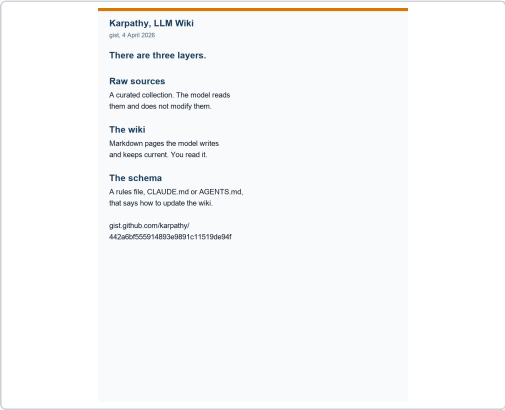


Term	What it adds
Document repository	files in folders, nothing more
Wiki	pages written for readers, linked by hand
Knowledge base	curated, reviewed entries
Link graph	which page links to which
Knowledge graph	typed relations: uses, estimated on, contradicts
GraphRAG	extracts a graph from text and retrieves over its communities

Typed relations answer questions like: which results depend on this dataset?

Source: Edge et al. (2024).

Andrej Karpathy's LLM Wiki



Raw sources

Papers, articles, notes. The model reads them and never edits them.

Wiki

Markdown pages the model writes and keeps current: summaries, entities, concepts, links.

Schema

A rules file that says how the wiki is organised and how to update it.

Three layers, from the gist of 4 April 2026.

The model does the bookkeeping that usually makes people give up on a wiki.

Source: Karpathy (2026).

Ingest, query, lint

Ingest

Read one new source. Write its summary page. Update every page it touches. Log what changed.

Query

Ask the wiki, not the raw pile. A good answer can be filed back as a page.

Lint

Look for contradictions, stale claims, orphan pages, missing links, concepts without a page.

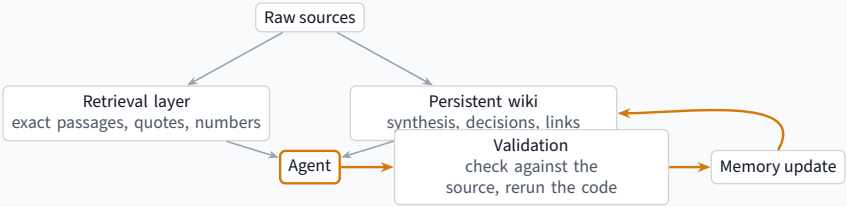
Promotion: a result checked in a session becomes a page, a decision, or a skill.

One paper, five edits

summary page (new); staggered DiD concept page (estimator added); employment entity page (new estimate); open questions (conflict with an earlier result flagged); index (two new rows).

Source: Karpathy (2026).

Hybrid RAG + LLM Wiki



The wiki answers: what do we know, and what did we decide? Retrieval answers: where exactly does the source say it?

Orient with the wiki. Go back to the source for any number you publish.

Comparing architectures, and firms

Design	Setup	Keeping current	Retrieval	Provenance	Reuse
Full prompt	none	resend everything	complete, if it fits	the source itself	none
RAG	index pipeline	re-index changed files	depends on chunking	passage	low
Knowledge base	curation time	manual review	high on covered topics	entry	high
GraphRAG	extraction, many tokens	re-extract	good on global questions	entity, indirect	medium
LLM Wiki	a folder and a schema	the agent, at each ingest	good for synthesis	links to raw sources	high
Hybrid	highest	both	best, if maintained	both	high

Qualitative comparison by the author.

In firms

Glean and Microsoft 365 Copilot index company documents and answer within each user's existing permissions. Meta has described an internal organisational second brain.

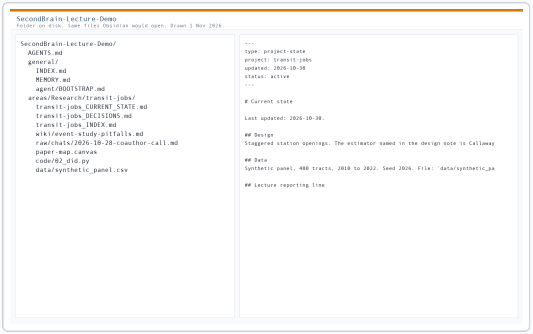
The hard part is governance: who may see what, stale pages that look authoritative, memory that outlives the reason it was written.

Source: Glean (2026); Microsoft (2026); Sengar et al. (2026).

Building one

Obsidian, Markdown, an agent, and Git

Why Obsidian?



Lecture vault on disk: file tree and the state file.

Source: Obsidian (2026).

Vault	a folder on your disk; Obsidian opens it
Markdown	plain text, readable without Obsidian
Local files	nothing leaves the machine unless you sync it
Plugins	add-ons; each one is code you chose to run
Agents	any agent with folder access reads the same files

Vaults, folders and Markdown

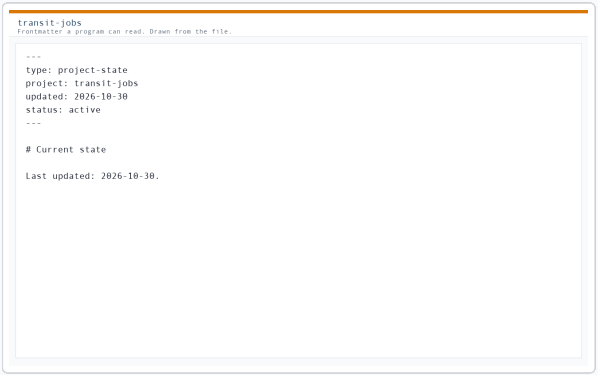
One folder per project.

The same file names in every project, so the agent knows where to look.

An INDEX file at each level: what is here, what to read first.

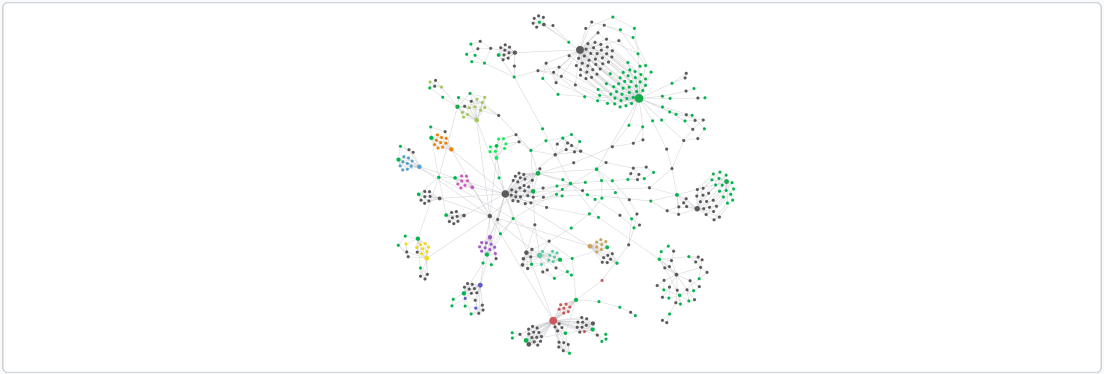
```
---
type: project-state
project: transit-jobs
updated: 2026-10-30
status: active
---
```

Frontmatter: fields a program can read. Obsidian shows them as Properties.



Frontmatter of the state file. A program can read these fields.

Links, backlinks and graph view



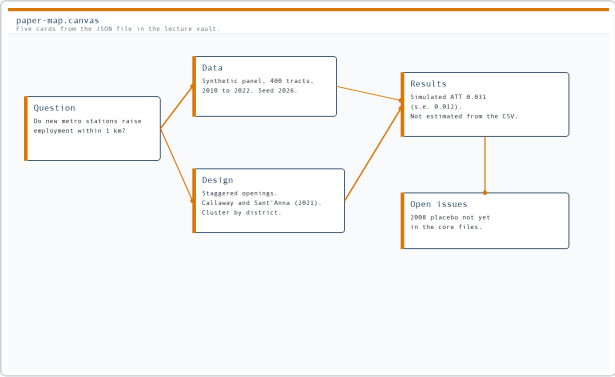
My vault in Obsidian graph view, July 2026. Colours are folders. Node labels are off.

`[[wikilinks]]`: type `[[DECISIONS]]` to link a note.

Backlinks: each note lists the notes that link to it.

A link says the notes are related. A knowledge graph also says how: uses, estimated on, contradicts.

Obsidian Canvas and plugins



`paper-map.canvas`: five cards drawn from the JSON in the lecture vault.

Source: Obsidian (2026); JSON Canvas (2026).

Canvas: a board of cards and arrows. Cards are notes, text or images. Saved as a `.canvas` file in the vault.

Three core plugins to turn on

Templates: the same skeleton for each new note.

Properties: frontmatter as editable fields.

Backlinks: who points here.

Each community plugin is third-party code with access to your vault.

Build your first brain



A first vault. Six files are enough.

README.md	what this vault is, for you and for the agent
INDEX.md	the map; read first
PROJECT_BRIEF	question, scope, people; changes rarely
CURRENT_STATE	where things stand today
NEXT_ACTIONS	the next three things to do
DECISIONS	date, decision, reason

Six files are enough for the first month. Add structure when a file gets too long.

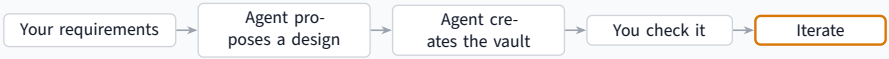
Connect an AI agent



Folder access: open the vault as the workspace, or add it as an extra folder.
Permissions: read freely; ask before writing; nothing outside the vault.
Instructions: `AGENTS.md` at the root points to `BOOTSTRAP.md`.
Bootstrap: read INDEX, then only the project you name.
Canonical memory: the Markdown files. Anything else is support.
Controlled writes: the agent proposes edits; Git shows the diff.

> **Reading is not remembering**
> The agent remembers only what someone wrote back into the files.

Let AI build your brain

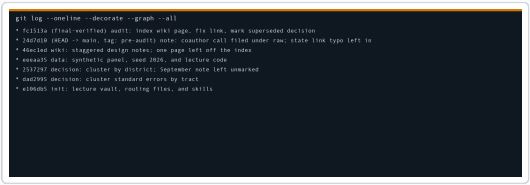



```
I am a PhD student in economics. Create an Obsidian vault in
~/Documents/MyBrain for my research. Put one folder per project
under projects/. In each project create PROJECT_BRIEF.md,
CURRENT_STATE.md, NEXT_ACTIONS.md and DECISIONS.md. Write
general/INDEX.md, an AGENTS.md that tells any agent to read
general/INDEX.md first, a BOOTSTRAP.md for opening a session and
a SESSION_CLOSE.md for closing one. Do not invent content: leave
placeholders. Show me the folder tree and wait for my approval
before writing any file.
```

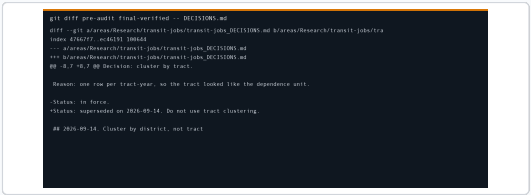
Prompt for students. The last sentence is the point: the tree, then approval.

Ask for the tree first. Then read what it wrote.

Git, GitHub and Obsidian



Git log, all branches. Audit on `final-verified`.



Diff of the decisions file after the audit.

Dropbox	syncs files across devices; keeps versions, without messages
Git	records each change with a message: diff, history, rollback
GitHub	hosts the repository; make it private for notes
Obsidian Git	commits and pushes from inside Obsidian

`.gitignore` keeps secrets, caches and large data out. Two machines editing the same note create a conflict you resolve by hand.

Source: Chacon and Straub (2014); GitHub (2026); Vinzent (2026).

My second brain

The vault I use every day, and how it is kept clean

My brain architecture

```
Marco/
├── README.md
├── general/          rules, routing, skills, templates
│   ├── agent/            BOOTSTRAP, SESSION_CLOSE, MEMORY_POLICY, WIKI_WORKFLOW
│   ├── capabilities/     skills/, integrations/
│   └── templates/        PROJECT_TEMPLATE/
├── areas/            Research, Grant, Teaching, Tesi, Other
└── .infra/           scripts, prose tools, integrations (hidden)
```

Checked against the live folder on 1 November 2026. No credential files in this tree.

One canonical memory

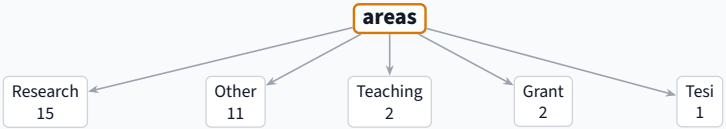
Markdown in `general/` and `areas/` is the source of truth. Tools, graphs and the local memory server are support layers. When they disagree, the file wins.

Where it lives

On Dropbox since 3 October 2026. New projects come from one script, `create_project.py`.

Source: Brain files `README.md`, `general/INDEX.md`, `general/GLOBAL_DECISIONS.md`.

One brain, many projects



Research: mafia-culture, mafia-incendi, fragmented-opposition. Other: brain-management. Teaching: AI_course.

```
| mafia-culture           | INDEX | CURRENT_STATE | AGENT_MEMORY | DECISIONS |
| mafia-incendi           | INDEX | CURRENT_STATE | AGENT_MEMORY | DECISIONS |
| fragmented-opposition   | INDEX | CURRENT_STATE | AGENT_MEMORY | DECISIONS |
```

Three rows. The real cells are links, shortened here. `general/MEMORY.md` is a routing table: one row per project, with links to INDEX, CURRENT_STATE, AGENT_MEMORY and DECISIONS. The agent opens one row, not thirty projects.

Anatomy of a project

Read at every start

INDEX: map and restart point.
CURRENT_STATE: where it stands.
NEXT_ACTIONS: what comes next.
AGENT_MEMORY: what the next agent must know.

Read when needed

PROJECT_BRIEF: question and scope.
DECISIONS: date, context, decision, reason.
TECHNICAL_NOTES: commands, bugs, fixes.
SOURCES: where facts come from.

History

LOG, by date. SESSION_SUMMARIES, one paragraph.
PROMPTS worth reusing.
Folders: `raw/`, `wiki/`, `skills/`.

```
## 2026-06-05 – Deterministic rerun belongs in a global skill
- Decision: capture the Google News + Media Cloud pattern as a
  reusable skill under general/capabilities/skills/.
- Reason: restartable for new keywords or periods, not one run.
```

Fragmented opposition, 5 June 2026. A procedure left the chat and became a skill.

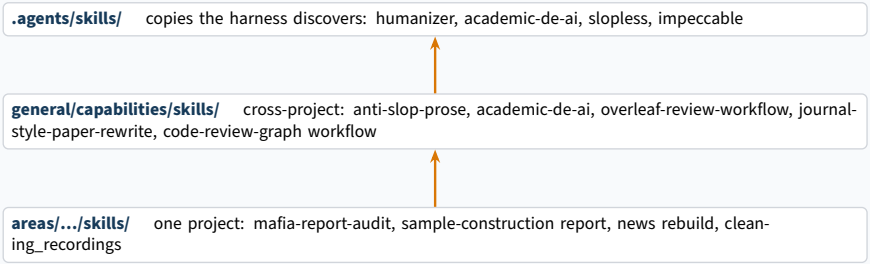
Separate files, separate jobs. The agent reads four short files instead of one long one.

My tool stack

Function	Tool	What it does	Status
Memory	Obsidian + Markdown	the canonical store	in daily use
Memory support	Memanto, local server	semantic recall over past notes	on-prem online, 1 Nov 2026
File access	Mirage	virtual filesystem, local only	documented
Code	code-review-graph, CodeGraph	code structure for review	documented
Context	Headroom	compresses long tool output; not memory	documented
Context	Caveman	terse agent replies, fewer tokens	active in this course workspace
Review	Econoclast	adversarial referee on empirical papers	documented
Writing	prose-check, Slopless, Fast-DetectGPT, Humanizer, academic-de-ai	style and AI-tell checks before text goes out	in use
Writing	Overleaf review workflow	reads Review Mode comments, edits in colour	in use
Maintenance	Sunday review agent	weekly vault audit	ran 11, 18, 25 Oct

Documented: listed in `TOOLING_AND_INTEGRATIONS.md`. Checked: seen running on the date shown. Memanto's local server answered on 1 November 2026. The other rows were not started for this slide.

Portable skills in my brain



Spotify SQLite mirror pipeline: written inside `mafia-culture/skills`, then promoted to `general/` once the project paths were removed.

The harness sees names and descriptions until a task needs the full skill.

My adaptation of the LLM Wiki



Raw: logs and sources, never deleted by a wiki pass. Distillation: the agent picks what is stable. Core: CURRENT_STATE, DECISIONS, AGENT_MEMORY, TECHNICAL_NOTES, patched first. Wiki: only long, stable pages. Index: every wiki page listed. An unlisted page is a defect.

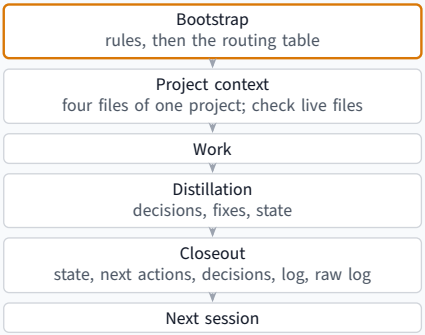
What I changed

Karpathy puts the synthesis in wiki pages. I put most of it in a few fixed project files, because those are what the agent reads at every start (decision of 27 August 2026).

The index rule came on 5 October 2026, after an audit found 22 of 141 wiki pages missing from their index.

Source: Brain files `WIKI_WORKFLOW.md`, `LLM_WIKI_PROMPT.md`, `MEMORY_POLICY.md`, `GLOBAL_DECISIONS.md`.

Bootstrap and session close




```
## Before Starting Work
Report four things:
1. what you understand about the project;
2. the current state;
3. the next concrete action;
4. any missing, stale, contradictory, or unclear memory files.
```

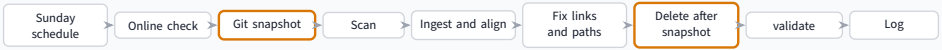
From `general/agent/BOOTSTRAP.md`. The agent reports this before editing. Until August 2026, every session read a 184-entry memory dump covering all projects. Since 27 August it reads the routing table and one project. Bootstrap must be O(project), not O(all memories).

A brain needs maintenance

Problem	In my vault	Lint check
Duplicates	8 duplicate wiki index files (18 Oct)	same content in two places
Stale instructions	23 live paths to the old Desktop vault (25 Oct)	paths against current rules
Contradictions	a stored memory still says the vault is on the Desktop (30 Oct)	support memory against Markdown
Broken links	32 from one orphan export (18 Oct)	every relative link resolves
Unindexed wiki	22 of 141 pages (5 Oct)	every wiki page in its INDEX
Superseded procedures	video-tool sections marked superseded (18 Oct)	skills that cite retired tools
Too much memory	a 184-entry dump read at every start (until Aug)	size of what bootstrap loads
Stale state	a state file last updated on 28 May	state date against the latest log

This is Karpathy’s lint step, run on a vault in daily use.

My Sunday maintenance



Run	What happened	What it did
11 Oct	9:02 run failed (Mac offline); rerun	raw already in core files; 0 unindexed wiki pages; restart-point fixes proposed, not applied
18 Oct	manual rerun	1,031 Markdown files, 28 projects, 144 wiki pages, 481 raw files; snapshot `3b61096`; 22 files deleted
25 Oct	8:57 run failed (Mac offline); rerun at about 10:55 with my approval	939 Markdown files; snapshot `939fc3c`; 23 old paths rewritten; 19 files deleted

Never read: credential files and `.env`. Each deleted file can be restored from the snapshot.

Two of three runs needed a manual restart. A schedule helps; it does not guarantee the run.

Source: Brain file `brain-management_LOG.md`; closeouts 2026-10-18 and 2026-10-25. The run is a scheduled agent. The scheduler is not named here.

Keeping your brain healthy

1. Save what the next session needs, not every conversation.
2. Update state and decisions at every close.
3. Audit on a schedule, and read the audit.
4. Keep the source next to each claim.
5. Commit before an agent edits; read the diff after.
6. Keep private material out of anything you share or sync.
7. Write repeated procedures as skills.
8. Let an agent do the maintenance, with a snapshot first.

JARVIS remembered because the script said so. Your assistant remembers what is in the files.

How useful an assistant is depends on the model, and just as much on the memory, tools and procedures around it.

Live: a second brain on this Mac

What we do

Open the vault. Start a session. Ask about a past decision. Create a project. Run a skill. Run the audit. Read the diff, commit, close.

What to look at

Which files the agent opens. Where its answer comes from. What changed on disk.

Live on screen share. Demo folder: `https://github.com/MarcoLeMoglie/Agentic_Economic_Research/tree/main/demos/lesson2`. If the agent fails, `git checkout final-verified` and the backup shots.

Before Lesson 3

Turn your `AGENTS.md` into a small vault.

Six files, one project.

Write one procedure you repeat as a `SKILL.md`.

Name, description, steps, how to check it worked.

Close one session by hand.

State, next actions, decisions. Lesson 3 runs steps like these without you.

References I

Agent Skills. Agent skills specification, 2026. URL https://agentskills.io/specification. Accessed 1 November 2026.

AGENTS.md. AGENTS.md, 2026. URL https://agents.md/. Accessed 1 November 2026.

American Economic Association. Data and code availability policy, 2026. URL https://www.aeaweb.org/journals/data/data-code-policy. Accessed 1 November 2026.

Anthropic. Effective context engineering for AI agents. Anthropic Engineering, 2025. URL https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

Anthropic. Prompt caching, 2026a. URL https://platform.claude.com/docs/en/build-with-claude/prompt-caching. Accessed 1 November 2026.

Anthropic. How Claude remembers your project, 2026b. URL https://code.claude.com/docs/en/memory. Accessed 1 November 2026.

Anthropic. Agent skills, 2026c. URL https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview. Accessed 1 November 2026.

Erik Brynjolfsson, Danielle Li, and Lindsey Raymond. Generative AI at work. *Quarterly Journal of Economics*, 140(2):889–942, 2025. doi: 10.1093/qje/qjae044.

Vannevar Bush. As we may think. *The Atlantic Monthly*, July 1945. URL https://www.theatlantic.com/magazine/archive/1945/07/as-we-may-think/303881/.

Brantly Callaway and Pedro H. C. Sant'Anna. Difference-in-differences with multiple time periods. *Journal of Econometrics*, 225(2):200–230, 2021.

Scott Chacon and Ben Straub. Pro git, 2014. URL https://git-scm.com/book/en/v2. Second edition.

Wesley M. Cohen and Daniel A. Levinthal. Absorptive capacity: A new perspective on learning and innovation. *Administrative Science Quarterly*, 35(1): 128–152, 1990.

References II

Cursor. Rules, 2026. URL https://cursor.com/docs/context/rules. Accessed 1 November 2026.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. arXiv:2404.16130, 2024. URL https://arxiv.org/abs/2404.16130.

Tiago Forte. *Building a Second Brain*. Atria Books, 2022.

Luis Garicano. Hierarchies and the organization of knowledge in production. *Journal of Political Economy*, 108(5):874–904, 2000.

GitHub. Setting repository visibility, 2026. URL https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings/setting-repository-visibility. Accessed 1 November 2026.

Glean. Work AI that works, 2026. URL https://www.glean.com/. Product page. Accessed 1 November 2026.

F. A. Hayek. The use of knowledge in society. *American Economic Review*, 35(4):519–530, 1945.

Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Chroma, July 2025. URL https://research.trychroma.com/context-rot.

JSON Canvas. JSON canvas specification, 2026. URL https://jsoncanvas.org/spec/1.0/. Accessed 1 November 2026.

Andrej Karpathy. LLM wiki. GitHub Gist, 4 April 2026, April 2026. URL https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 2020. URL https://arxiv.org/abs/2005.11401.

References III

Microsoft. Data, privacy, and security for Microsoft 365 Copilot, 2026. URL https://learn.microsoft.com/en-us/copilot/microsoft-365/microsoft-365-copilot-privacy. Accessed 1 November 2026.

Obsidian. Obsidian help, 2026. URL https://help.obsidian.md/. Accessed 1 November 2026.

OpenAI. Agent skills, 2026a. URL https://developers.openai.com/codex/skills. Accessed 1 November 2026.

OpenAI. Prompt caching, 2026b. URL https://platform.openai.com/docs/guides/prompt-caching. Accessed 1 November 2026.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3 (4):333–389, 2009.

Shaurya Sengar, Jason Nawrocki, Jay Shah, and Prashant Kommireddi. An organizational second brain: Building an AI that learns from experts. Engineering at Meta, 2 September 2026, September 2026. URL https://engineering.fb.com/2026/09/02/ml-applications/organizational-second-brain-ai-learns-from-experts/.

Vinzent03 Vinzent. Obsidian git, 2026. URL https://github.com/Vinzent03/obsidian-git. Accessed 1 November 2026.

James P. Walsh and Gerardo Rivera Ungson. Organizational memory. *Academy of Management Review*, 16(1):57–91, 1991.

Appendix

Not for the main hour. Read them after class.

Token cost of one request

$$\text{cost} = \text{input} \times p_{\text{in}} + \text{cached input} \times p_{\text{cache}} + \text{output} \times p_{\text{out}}$$

Prices per million tokens. Cached input is billed at a fraction of p_{in}.

Lesson 1 lists a few public prices. They move. Read the vendor page on the day you pay.

Source: Anthropic (2026a); OpenAI (2026b).

Caching versus a smaller context

Risks of persistent memory

Governance and access

Organisational memory and human capital

Reproducibility

A possible experiment

Checklists

A personal brain

One folder. An index. Four project files. A rules file. No secrets. A `.gitignore`. One skill. A closeout you have done once by hand.

Maintenance

Unindexed pages. Broken links. Decisions that contradict a later one. Duplicate notes. Raw material not distilled. Stale paths. What bootstrap loads. A snapshot before any deletion.

If the live demo fails